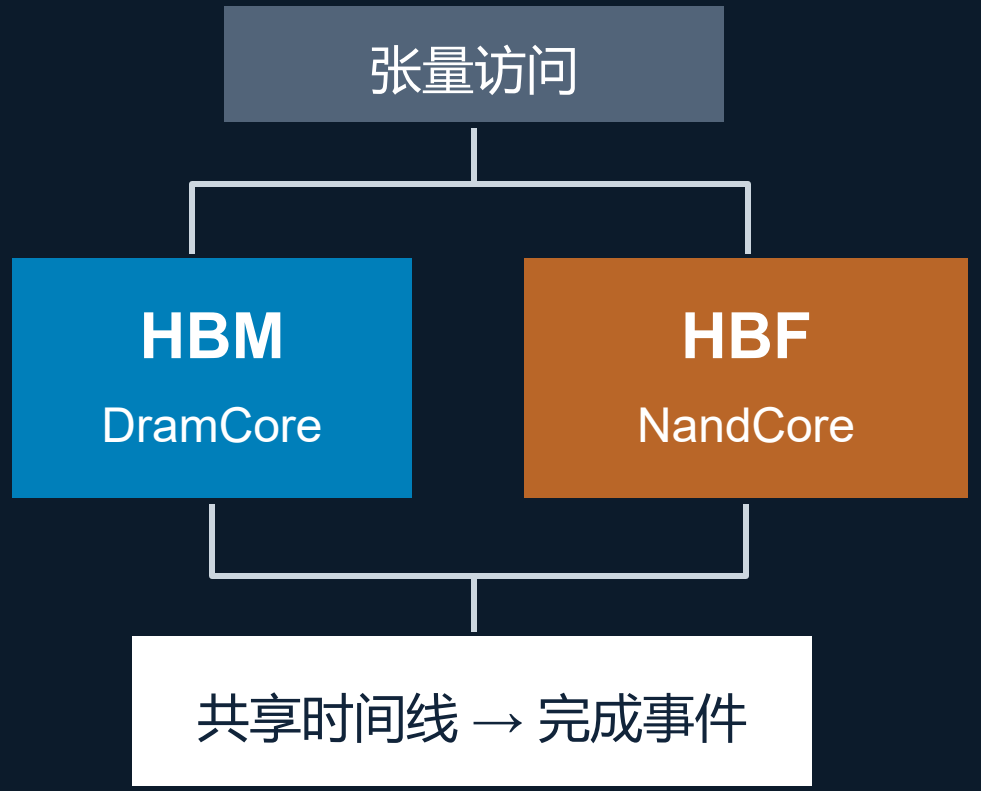
HBM / HBF 建模流程

同一条访存请求，两套物理规则

地址、介质状态与资源竞争
怎样共同决定完成时间



仿真器的完整链路

先生成访问，再让有状态的物理 Core 计算实际完成时间



规划阶段的预览值，会在任务实际到达时由 Core 结果替换

源码：planner.py · data_motion.py · event_kernel.py · reporting.py

两种介质的建模对象

共用访问契约，分别维护介质自己的状态

HBM　高带宽 DRAM

DramConfig → DramCore

服务粒度：burst

JEDEC HBM 路径固定 32 B

状态：打开的行、命令与刷新

HBF　NAND 页服务

NandConfig → NandCore

服务粒度：page

论文原生配置采用 4 KiB 页

状态：plane、页缓冲、传输

输入：操作 / 地址 / 字节数 / 到达时间　→　输出：完成时间

源码：memory_types.py · dram_core.py · nand_core.py　|　具体数值属于相应模式或配置

配置先定义物理约束

访问需求、数据位置与硬件参数分别进入模型

工作负载

模型结构、精度

Batch、上下文长度

TP / EP / PP 等并行配置

放置与生命周期

每个 rank 的存储端点

权重 / KV / 激活

分配、释放、缓冲区代次

硬件输入

容量、通道、bank / plane

方向带宽、时延、队列

控制模式、协议与策略



容量决定能否放下；地址与时序决定怎样服务；峰值带宽限定传输上限

源码：memory_types.py · physical_contract.py · planner.py

张量访问到物理地址

每条访问都要说明：在哪个端点、访问哪些字节、何时到达

权重读取

KV 历史读 / 新 token 写

激活与中间结果读写

→

operation　physical_owner　byte_count

address　或　buffer_id + offset_bytes

分配器解析地址，同时校验容量、分配代次与访问范围

HBM：拆成 burst

映射 lane / bank / row

HBF：拆成 page

映射 channel / plane / page

逻辑字节 ≠ 物理流量

对齐、整 burst、页传输与 RMW 都可能放大流量

源码：data_motion.py:1151 · memory_mapping.py:172,212 · memory_allocator.py

HBM 地址映射与并行度

地址落在哪个 lane、bank 与 row，决定并行机会和冲突位置

Stack	→	Channel × 32	→	PC × 2	→	Bank → Row
HBM 堆		HBM4 每堆		每个物理 channel		含 SID / bank group

SID 已体现在 bank 地址中，容量计算不再重复乘一遍 die 数

bank_interleaved

同 SID 内轮换 bank group

先分散 lane → bank → column → row

0 → 8 → 1 → 9 → 2 → 10 → …

示意：每组 8 个 bank；持续访问不再一直落在同组

源码：hbm_geometry.py:102,137 · memory_mapping.py:18,47 · _hbm_control.py:60

HBM 的两种控制模式

模式决定建模精度；执行加速不会自动改变控制模式

analytical

简化 DRAM 服务模型

访问 → 简化行状态 / 传输约束 → 完成

不模拟刷新与 tRAS；不能套用完整命令时序解释

jedec_hbm

显式正常工作命令控制器

访问 → ACT / PRE / RD / WR / REF → 完成

行状态、共享命令总线、PC 数据总线

列间隔、ACT 速率、刷新、读写方向切换

后续 HBM 时序图解释 jedec_hbm 路径；范围为正常工作命令，保留模型抽象

源码：dram_core.py:90,423,483 · _hbm_control.py:1

HBM 行状态与命令链

同样的读取字节数，可能经历不同命令与等待

行命中
目标行已打开

RD → RL → DQ 数据传输

bank 关闭
先打开目标行

ACT → tRCD → RD → RL → DQ 数据传输

行冲突
先关旧行，再开新行

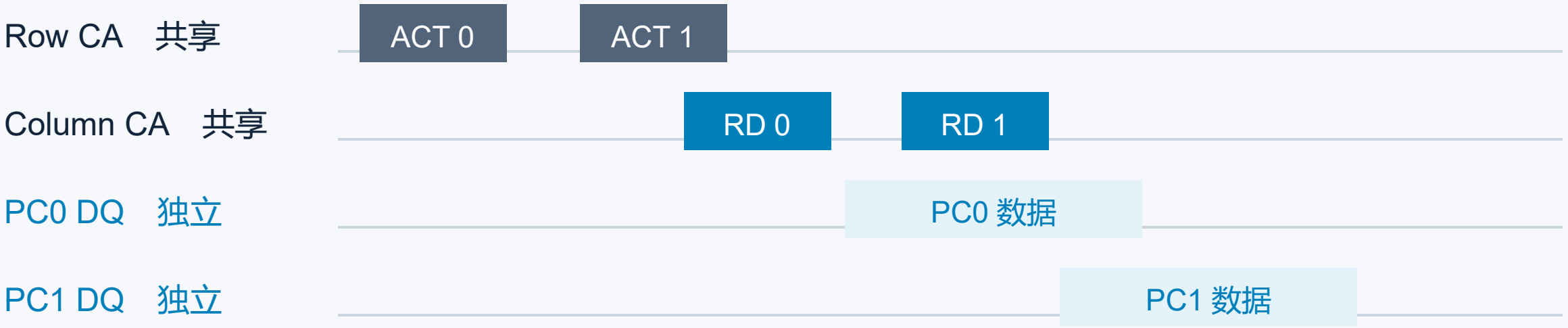
PRE → tRP → ACT → tRCD → RD → RL → DQ 数据传输

写操作对应 WR / WL；请求完成取所有 burst 数据结束时间的最大值

源码：_hbm_control.py:145,183,490,547 | 概念顺序图，非按时长比例；省略精细时钟锚点

HBM 共享资源时间线

同一 channel 的两个 PC 共享命令总线，各自拥有数据总线



时间向右 → 示意共享与重叠，长短不代表数值

发射 = 时钟对齐后的 max（请求、bank、总线、时序约束）

tCCD / tRRD / tFAW / 方向切换；前瞻 PRE 与 all_bank 刷新也进入时间线

HBF 几何与资源映射

论文原生配置：资源数由 stack / die / plane 几何产生

每个 Stack
一组真实 die

→

16 Die
各自映射内部 channel

→

25 Plane / Die
独立阵列服务资源

→

Block → Page
4 KiB 页

连续页先条带到 die 通道，再轮转 plane，之后推进 block 内 page

7 堆 HBF 配置

112 个内部通道

7 × 16

2,800 个阵列单元

7 × 16 × 25

传输带宽分别限制内部通道和 host 出口，阵列延迟仍然存在

源码：exploring_hbf_paper_config.py:153,289 · memory_mapping.py:124 · memory_types.py:813

HBF 读请求的三段服务

队列准入 → 按页拆分和映射 → 查询该 plane 的两个读缓冲槽

缓存未命中

PAGE_READ
plane 阵列读取

→

INTERNAL_TRANSFER
内部通道搬入 slot

→

HOST_TRANSFER
host 出口返回数据

缓存命中

等待 slot.ready 与 host 资源 → HOST_TRANSFER

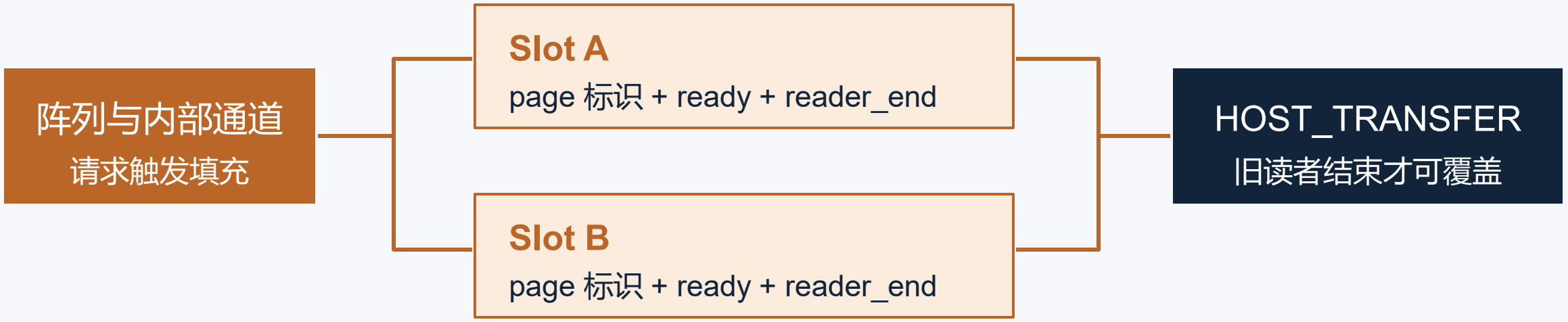
同资源排队，独立资源可按依赖重叠

一条请求的完成时间 = 各页完成时间的最大值；延迟仍包含准入等待

源码：nand_core.py:537,549,553,1900,1919,1923,1937 | generic_nand 双缓冲读路径

每个 plane 的双缓冲

填充完成前不可读；旧读者结束前不可覆盖该槽



每堆 3.125 MiB

16 × 25 × 2 × 4096 = 3,276,800 B

媒体页缓冲与 Plus 的每堆 40 MB 中间 SRAM 分开

填充由读请求驱动；当前基线未建自主预测预取控制器

源码：nand_core.py:647,666,1900,1925 · exploring_hbf_paper_config.py:276,325

HBF 写入与协议边界

完整页写入的完成点，落在 PAGE_PROGRAM 结束

HOST_TRANSFER
数据进入接口

→

INTERNAL_TRANSFER
内部搬运

→

PAGE_PROGRAM
阵列编程结束才完成

部分页写入　reject　或　付费 read-modify-write

RMW 支付旧页读取、内部搬运与新页 program

显式擦除　按物理 block 去重，占用阵列资源

论文原生基线：generic_nand

直接地址页服务，无默认 FTL / GC 或自动擦除；OCP 约束属于另一个协议模式

源码：nand_core.py:1861,1958,1964,1982 · memory_mapping.py:216 · memory_types.py:739

跨端点数据搬运与依赖

COPY 展开为源读取、链路传输、目标写入，各自进入物理资源时间线

源 Core 读

源介质的时序与流量

→

链路传输

带宽、共享与排队

→

目标 Core 写

目标写完成后才完成搬运

独立请求与独立资源可以重叠；同资源按其可用时间等待

同一地址的读写依赖

RAW / WAR / WAW

先写后读、先读后写、连续写必须保留顺序

任务实际 dispatch

提交有状态 Core → 完成事件 → 后继任务

部分原生 kernel 采用计算 / 内存跨度的执行包络，仍属于任务级模型抽象

源码：data_motion.py:1261,1330,1730,2573 · memory_transfer.py:263 · event_kernel.py:685

输出指标与瓶颈判断

把完成时间、字节流量与资源忙碌放在一起阅读

Core 结果

完成时间 / 流量

等待 / 资源占用

HBM：行命中 / 冲突与命令计数

ACT / PRE / RD / WR / REF，CA 与 DQ 忙碌

HBF：缓存与三段资源

命中、实际读页 / program、array / internal / host

系统：事件完成与可见 Token

整体 makespan、吞吐、TTFT / TPOT（按统计边界）

请求延迟 = 完成 − 到达 | 有效带宽 = 逻辑字节 / 相应时间窗

service_ns 累加的服务跨度可能重叠；资源忙碌看 resource_totals，总耗时看 makespan

源码：memory_types.py:899,911 · memory_service.py:26–84 · reporting.py:632,2113,2174

快速仿真与物理语义

降低模拟器执行开销，保留所选模型的物理时序规则

HBM 数值路径

Numba、数组状态、批量提交

仍计算命令约束与资源预约

NAND 逐页编译路径

编译页循环，减少对象和明细开销

保留缓存访问、填充与读者寿命

相同的介质模型

同一组输入与资源约束

有适用条件，配有语义对照测试

加速路径不适用时执行参考实现；HBM 不会因此从 jedec_hbm 切换到 analytical

源码：dram_core.py:423,527 · _compiled_hbm.py:92 · _nand_acceleration.py:1,59

一组配置输入示例

项目中的论文原生对照配置；以下是输入参数，未展示本次性能测量

HBM4 配置

8 堆 HBM / GPU

读 / 写接口预算：12.8 TB/s

HBM4 标准参考组织与正常工作时序

采用显式 JEDEC 控制路径

HBF 配置

7 堆 HBF + 1 堆 HBM / GPU

HBF 读 11.2 TB/s；写 112 GB/s

读页 1 µs；program 100 µs

16 die × 25 plane / 堆；4 KiB 页

显式研究假设

HBM4 的 tRRD_S / L = 4 / 6 ns、tFAW = 20 ns，借用 HBM3E 参考

NAND：256 页 / block、erase 2 ms、每 die 内部通道 100 GB/s 等为本地选择

源码：scripts/exploring_hbf_paper_config.py:144,207,232,272 | 带宽均为聚合上限，非持续吞吐

理解模型的四个抓手

请求 → 地址 → 状态 → 时间

每个性能变化，都应沿这条链路追溯原因

HBM 行状态、命令与总线

HBF 页、双缓冲与三段传输

配置与映射：memory_types.py / memory_mapping.py

介质与系统：_hbm_control.py / nand_core.py / data_motion.py / reporting.py